

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1408

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

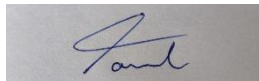
SHORT ANSWER TEST

Code of Conduct:

I Tamil Elakiya S (192524443) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:

id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+
*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:

id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q15. Given the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Q17. Construct the augmented grammar for:

$S \rightarrow a$

Q18. Find the closure of the LR(0) item:

$S' \rightarrow \bullet S$

for the grammar:

$S \rightarrow a$

Q19. Check whether the grammar is ambiguous:

$E \rightarrow E + E$

$E \rightarrow id$

using the string:

id + id + id

Q20. Construct a leftmost derivation for the string:

id + id

using the grammar:

$E \rightarrow E + E$

$E \rightarrow id$

Q21. Apply left factoring to the grammar:

$S \rightarrow aA$

$S \rightarrow aB$

$A \rightarrow b$

$B \rightarrow c$

Q22. Find FIRST(A) for the grammar:

$A \rightarrow x$

$A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Show the table entries for the terminals:

a, b, \$

name: Tamil Elakiya S

Date: 6/7/26

reg. NO: 192524443

course: compiler Design

Short Answer Test

course

CO-2

code: CSA1408

Q1. Token stream:

$id + id$

The parser validates the structure of expression as:

$E \rightarrow E + E \mid id$

The parser checks if the input matches the Grammar. and if the Grammar is matched, the parser validates the structure of expression.

Q2. $S \rightarrow aS \mid b$

The string: 'aaab'

$S \rightarrow aS$

then, $S \rightarrow aS \Rightarrow S \rightarrow aS \Rightarrow S \rightarrow b$

So, The grammar can produce the string 'aaab' as described above.

flow: $S \rightarrow aS$
 $S \rightarrow a\bar{a}S$
 $S \rightarrow aa\bar{a}S$
 $S \rightarrow aaab$

Q3. Remove left recursion:

$A \rightarrow Aa \mid b$

After removing left recursion:

$A \rightarrow bA'$

$A' \rightarrow aA' \mid \epsilon$

Q4. FIRST(S):

Given:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

$FIRST(S) = \{a, b\}$

5. Given Grammar:

$$S \rightarrow Sa | b$$

The grammar is not suitable for top-down parsing as Recursive parsing / LL(1) cannot handle the left Recursion.

If the left recursion is removed, then it is suitable for top-down parsing.

6. Given:

$$S \rightarrow \text{if } E \text{ then } S$$

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$

Applying left factoring:

$$S \rightarrow \text{if } E \text{ then } S S'$$

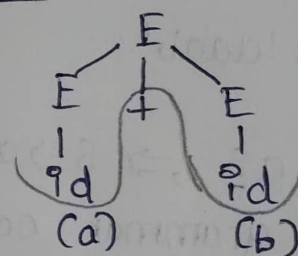
$$S' \rightarrow \text{else } S \mid \epsilon$$

7. Parse tree for $a+b$:
using the grammar:

$$E \rightarrow E + E$$

$$E \rightarrow id$$

Parse tree:



8. Recursive descent procedures:

$$S \rightarrow aA$$

$$A \rightarrow b$$

```
S() {
    if (Input == 'a') {
        input++;
        A();
    }
    else {
        error();
    }
}
```

```
A() {
    if (Input == 'b') {
        input++;
    }
    else {
        return;
    }
}
```

```
main() {
    S();
    if (Input == '$') {
        printf("Parsing successful");
    }
}
```

$S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$

Predictive Parsing table:

	a	b	\$
S	$S \rightarrow AB$		
A	$A \rightarrow a$		
B		$B \rightarrow b$	

10. First three actions: $id + id$

Stack Input Buffer action
 $\$$ $id + id \$$ reduce $E \rightarrow id$
 $\$ id$

stack	Input Buffer	action
$\$$	$id + id \$$	shift
$\$ id$	$+ id \$$	reduce $E \rightarrow id$
$\$ E$	$+ id \$$	shift

11. Grammar:

$E \rightarrow E + E$

$E \rightarrow id$

First handle for the above grammar:

' id ' is the first handle

12. Operator precedence relations:

	id	$+$	$*$	$\$$
id	$-$	$\rightarrow$	$\rightarrow$	$\rightarrow$
$+$	$<$	$\rightarrow$	$<$	$\rightarrow$
$*$	$<$	$\rightarrow$	$\rightarrow$	$\rightarrow$
$\$$	$<$	$<$	$<$	$=$

13. Expression:

$id * id$

operator precedence parsing handles as.

$\$ < \underline{id} \rightarrow * < id \rightarrow \$$

The parser scans until it find $\rightarrow$, if found it reduces as follows:

$$\hookrightarrow \$ \langle E * \underline{id} \rangle \$$$

$\downarrow$
 E

$$\hookrightarrow \$ \langle E * E \rangle \$$$

$$\hookrightarrow \$ \langle E \rangle \$$$

$$E \rightarrow E * E$$
$$E \rightarrow id \mid \epsilon$$

It reaches the start symbol and the parser accepts.

14. LR Parser:

LR Parser is a non-recursive, bottom-up parser.

Where, L $\rightarrow$ The input is scanned from left to right

R $\rightarrow$ construct the right most derivation in reverse

$$S \rightarrow aA$$

$$A \rightarrow b$$

Process:

$\hookrightarrow \$$ - shift a

$\hookrightarrow \$a$ - shift b

$\hookrightarrow \$ab$ - reduce $A \rightarrow b$

$\hookrightarrow \$aA$ - reduce $S \rightarrow aA$

$\hookrightarrow \$S$ - Accept

15. $S \rightarrow aA$ $A \rightarrow b$

Steps involved in constructing SLR parsing table:

* Remove left recursion or left factoring if present.

* compute First(), Follow().

* construct augmented grammar.

* Adding '.' After every non-terminal in each expression.

* Find Goto function.

* Table for Action and Goto.

* Then shift reduce parser table.

FOLLOW(A):

$$\text{Follow}(A) = \{\text{First}(B)\}$$

$$\therefore \text{Follow}(A) = \{b\}$$

Grammar:

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$\therefore \text{First}(B) = \{b\}$$

17. augmented grammar for: $S \rightarrow a$

$$S' \rightarrow S$$

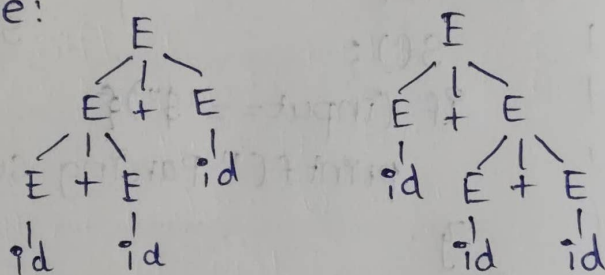
$$S \rightarrow .a$$

18. closure of LR(0):

$$S' \rightarrow .S = \{ S' \rightarrow .S, S \rightarrow .a \}$$

19. $E \rightarrow E + E$; string: $\text{id} + \text{id} + \text{id}$
 $E \rightarrow \text{id}$

The grammar is ambiguous as it has more than one parse tree:



20. leftmost derivation: $\text{id} + \text{id}$

$$E \rightarrow E + E ; E \rightarrow \text{id}$$

$$E \rightarrow \text{id} + E$$

$$E \rightarrow \text{id} + \text{id}$$

Grammar:

$$E \rightarrow E + E$$

$$E \rightarrow \text{id}$$

21. left factoring:

$$S \rightarrow aS'$$

$$S' \rightarrow A | B$$

$$A \rightarrow b$$

$$B \rightarrow c$$

grammar:

$$S \rightarrow aA$$

$$S \rightarrow aB$$

$$A \rightarrow b$$

$$B \rightarrow c$$

22. FIRST(A) :

$$\text{FIRST}(A) = \{x, y\}$$

Grammar:

$$A \rightarrow x$$

$$A \rightarrow y$$

23. Check if Grammar is suitable for Predictive parsing:

The Grammar is not suitable for Predictive parsing as the given grammar is of left factoring.

$$S \rightarrow aA \mid aB$$

$$A \rightarrow b$$

$$B \rightarrow c$$

↳ It is suitable only after removing left factoring.

24. Procedures:

```
S() {  
    if (input == 'a') {  
        input++;  
    }  
    S();  
}  
if (input == 'b') {  
    input++;  
}  
}
```

```
|  
| main() {  
|     S();  
|     if (input == '$') {  
|         printf("Parsing successful");  
|     }  
| }
```

Grammar:

$$S \rightarrow aS$$

$$S \rightarrow b$$

25. Predictive Parsing table:

	a	b	\$
S	$S \rightarrow aS$	$S \rightarrow b$	

Grammar:

$$S \rightarrow aS$$

$$S \rightarrow b$$